

Preentrega Backend con Java

Jose Miguel Paz Portilla

6 de octubre de 2026

Índice

1. Creación del proyecto	3
1.1. build_new_project.sh	3
2. Ejecución del proyecto	6
2.1. run_project.sh	6
3. Código	7
3.1. Main.java	7
3.2. Validador.java	10
3.3. Producto.java	14
3.4. ProductoService.java	17
3.5. ProductoNoEncontradoException.java	20
3.6. StockInsuficienteException.java	21
3.7. MenuProducto.java	22
4. Corridas	26

1. Creación del proyecto

El proyecto por problemas de velocidad del ordenador lo realice desde la terminal de linux. Para crear un nuevo proyecto java en desde la terminal creo un script `build_new_project.sh` el cual crea las carpetas como lo necesita maven, crea las carpetas `ui`, `util`, `model`, `exception` y `service`. También crea un `Main.java` con un `Hola mundo` de prueba.

1.1. build_new_project.sh

Listing 1: Script de automatización `build_new_project.sh`

```
1  #!/bin/bash
2
3  # Detener el script inmediatamente si ocurre algun error
4  set -e
5
6  # =====
7  # VALIDACION DE ARGUMENTOS DE LINEA DE COMANDO
8  # =====
9  # Si el primer argumento ($1) esta vacio, muestra un error
   explicativo y sale.
10 if [ -z "$1" ]; then
11     echo "Error: No especificaste el nombre del proyecto."
12     echo "Uso correcto: $0 <nombre_del_proyecto>"
13     echo "Ejemplo: $0 preentrega"
14     exit 1
15 fi
16
17 # Asignamos el primer argumento de la terminal a nuestra variable
18 NUM_EJERCICIO="$1"
19
20 # Definicion de rutas estandar de Maven usando la variable
21 PACKAGE_DIR="src/main/java/com/josepaz/$NUM_EJERCICIO"
22 TEST_DIR="src/test/java/com/josepaz/$NUM_EJERCICIO"
23 RESOURCES_DIR="src/main/resources"
24
25 echo "=== 1. Generando estructura de carpetas estandar de Maven y
   paquetes internos para: $NUM_EJERCICIO ==="
26 # Creamos la ruta base del paquete y del entorno de pruebas
27 mkdir -p "$PACKAGE_DIR"
28 mkdir -p "$TEST_DIR"
29 mkdir -p "$RESOURCES_DIR"
30
31 # Creamos las subcarpetas de arquitectura dentro de la carpeta del
   ejercicio
32 mkdir -p "$PACKAGE_DIR/exception"
33 mkdir -p "$PACKAGE_DIR/model"
34 mkdir -p "$PACKAGE_DIR/service"
35 mkdir -p "$PACKAGE_DIR/util"
36 mkdir -p "$PACKAGE_DIR/ui"
37
38 echo "=== 2. Verificando archivo Java principal ==="
```

```

39 # Solo crea el Main.java de ejemplo inicial si el archivo NO existe
40 .
41 if [ ! -f "$PACKAGE_DIR/Main.java" ]; then
42     echo "Creando archivo Java de ejemplo (Main.java)..."
43     cat << EOF > "$PACKAGE_DIR/Main.java"
44 package com.josepaz.$NUM_EJERCICIO;
45
46 public class Main
47 {
48     public static void main(String[] args)
49     {
50         System.out.println("Hola Mundo desde Main.java en Maven (
51             $NUM_EJERCICIO)");
52     }
53 }
54 EOF
55 else
56     echo "El archivo Main.java ya existe. Respetando tu codigo
57         actual."
58 fi
59
60 echo "=== 3. Generando archivo pom.xml ==="
61 cat << EOF > pom.xml
62 <?xml version="1.0" encoding="UTF-8"?>
63 <project xmlns="http://apache.org"
64     xmlns:xsi="http://w3.org"
65     xsi:schemaLocation="http://apache.org http://apache.org">
66     <modelVersion>4.0.0</modelVersion>
67
68     <groupId>com.josepaz</groupId>
69     <artifactId>hola-mundo-terminal</artifactId>
70     <version>1.0-SNAPSHOT</version>
71
72     <properties>
73         <maven.compiler.source>21</maven.compiler.source>
74         <maven.compiler.target>21</maven.compiler.target>
75         <project.build.sourceEncoding>UTF-8</project.build.
76             sourceEncoding>
77     </properties>
78
79     <build>
80         <plugins>
81             <!-- Plugin para compilar con tu version Java 21 -->
82             <plugin>
83                 <groupId>org.apache.maven.plugins</groupId>
84                 <artifactId>maven-compiler-plugin</artifactId>
85                 <version>3.13.0</version>
86                 <configuration>
87                     <source>21</source>
88                     <target>21</target>
89                 </configuration>
90             </plugin>

```

```

88      <!-- Plugin Exec apuntando a tu clase dinamica com.
          josepaz.$NUM_EJERCICIO.Main -->
89      <plugin>
90          <groupId>org.codehaus.mojo</groupId>
91          <artifactId>exec-maven-plugin</artifactId>
92          <version>3.5.0</version>
93          <configuration>
94              <mainClass>com.josepaz.${NUM_EJERCICIO}.Main</
                  mainClass>
95          </configuration>
96      </plugin>
97  </plugins>
98  </build>
99 </project>
100 EOF
101
102 echo "=== 4. Limpiando y compilando el proyecto ==="
103 mvn clean compile
104
105 echo "=== 5. Ejecutando la clase Main de $NUM_EJERCICIO ==="
106 mvn exec:java

```

Este script debe tener permiso de ejecución que se otorga del siguiente modo:
chmod +x build_new_project.sh

Luego se ejecutó para crear el proyecto como se muestra en la Figura 1.

```

josepaz@josepaz-Lenovo-IdeaPad-S145-15IGM:~/JAVA/java-preentrega$ ./build_new_project.sh preentrega
=== 1. Generando estructura de carpetas estándar de Maven y paquetes internos para: preentrega ===
=== 2. Verificando archivo Java principal ===
Creando archivo Java de ejemplo (Main.java)...
=== 3. Generando archivo pom.xml ===
=== 4. Limpiando y compilando el proyecto ===
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.josepaz:hola-mundo-terminal >-----
[INFO] Building hola-mundo-terminal 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ hola-mundo-terminal ---
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ hola-mundo-terminal ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 0 resource
[INFO]
[INFO] --- maven-compiler-plugin:3.13.0:compile (default-compile) @ hola-mundo-terminal ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 1 source file with javac [debug target 21] to target/classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 2.502 s
[INFO] Finished at: 2026-10-01T00:59:38-03:00
[INFO]
[INFO] -----
[INFO]
=== 5. Ejecutando la clase Main de preentrega ===
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.josepaz:hola-mundo-terminal >-----
[INFO] Building hola-mundo-terminal 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- exec-maven-plugin:3.5.0:java (default-cli) @ hola-mundo-terminal ---
¡Hola Mundo desde Main.java en Maven (preentrega)!
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 0.874 s
[INFO] Finished at: 2026-10-01T00:59:42-03:00
[INFO]
josepaz@josepaz-Lenovo-IdeaPad-S145-15IGM:~/JAVA/java-preentrega$

```

Figura 1: Creación de un nuevo proyecto java

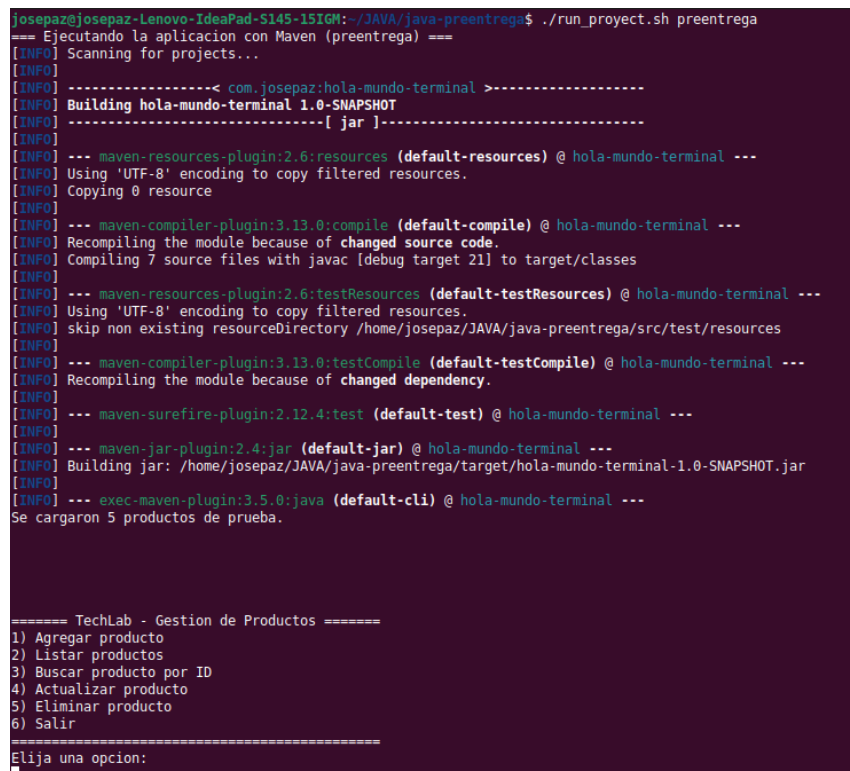
2. Ejecución del proyecto

2.1. run_project.sh

Listing 2: Script de ejecución run_project.sh

```
1 #!/bin/bash
2
3 # Detener el script inmediatamente si ocurre algun error
4 set -e
5
6 # Verificamos si pasaste el argumento por la terminal
7 if [ -z "$1" ]; then
8     echo "Error: No especificaste el proyecto a ejecutar."
9     echo "Uso correcto: $0 <nombre_del_proyecto>"
10    echo "Ejemplo: $0 preentrega"
11    exit 1
12 fi
13
14 # Asignamos el argumento a una variable clara
15 EJERCICIO="$1"
16
17 echo "=== Ejecutando la aplicacion con Maven ($EJERCICIO) ==="
18 mvn package exec:java -Dexec.mainClass="com.josepaz.${EJERCICIO}.
    Main"
```

Para ejecutarlo una vez que se realizó el proyecto se debe ejecutar como se muestra en la Figura 2.



```
josepaz@josepaz-Lenovo-IdeaPad-S145-15IGM: ~/JAVA/java-preentrega$ ./run_project.sh preentrega
=== Ejecutando la aplicacion con Maven (preentrega) ===
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.josepaz:hola-mundo-terminal >-----
[INFO] Building hola-mundo-terminal 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ hola-mundo-terminal ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 0 resource
[INFO]
[INFO] --- maven-compiler-plugin:3.13.0:compile (default-compile) @ hola-mundo-terminal ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 7 source files with javac [debug target 21] to target/classes
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ hola-mundo-terminal ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory /home/josepaz/JAVA/java-preentrega/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.13.0:testCompile (default-testCompile) @ hola-mundo-terminal ---
[INFO] Recompiling the module because of changed dependency.
[INFO]
[INFO] --- maven-surefire-plugin:2.12.4:test (default-test) @ hola-mundo-terminal ---
[INFO]
[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ hola-mundo-terminal ---
[INFO] Building jar: /home/josepaz/JAVA/java-preentrega/target/hola-mundo-terminal-1.0-SNAPSHOT.jar
[INFO]
[INFO] --- exec-maven-plugin:3.5.0:java (default-cli) @ hola-mundo-terminal ---
Se cargaron 5 productos de prueba.

===== TechLab - Gestion de Productos =====
1) Agregar producto
2) Listar productos
3) Buscar producto por ID
4) Actualizar producto
5) Eliminar producto
6) Salir
=====
Elija una opcion:
```

Figura 2: Ejecución del proyecto java

3. Código

3.1. Main.java

Listing 3: Clase Main.java (Pre-entrega)

```
1 package com.josepaz.preentrega;
2
3 import java.util.Scanner;
4
5 import exception.ProductoNoEncontradoException;
6 import exception.StockInsuficienteException;
7 import model.Producto;
8 import service.ProductoService;
9 import ui.MenuProducto;
10 import util.Validador;
11
12 /**
13  * Clase principal que actua como el Orquestador o Controlador
14  * General de la aplicacion.
15  * Inicia el programa, configura los datos de prueba y maneja el
16  * bucle principal de ejecucion.
17  */
18 public class Main
19 {
20     public static void main(String[] args)
21     {
22         // =====
23         // 1. CONFIGURACION E INYECCION DE DEPENDENCIAS
24         // =====
25         // Creamos las dependencias clave una sola vez en el punto
26         // de entrada de la app
27         // y las compartimos (inyectamos) para evitar multiples
28         // instancias innecesarias.
29         ProductoService service = new ProductoService();
30         Scanner sc = new Scanner(System.in);
31
32         // Pasamos el Scanner y el Servicio al menu para que
33         // trabaje sobre las mismas instancias
34         MenuProducto menu = new MenuProducto(sc, service);
35
36         // Llenamos el inventario inicial con productos precargados
37         // para facilitar las pruebas
38         cargarDatosDePrueba(service);
39
40         // Variable local para almacenar y evaluar la decision del
41         // usuario en cada iteracion
42         int opcion;
```

```

41 // El programa seguira corriendo de forma interactiva hasta
42 // que el usuario digite la opcion 6.
43 do
44 {
45     // Pinta las opciones disponibles en la terminal
46     menu.mostrarMenu();
47
48     // Lee la entrada numerica de forma segura delegando la
49     // validacion a la clase utilitaria
50     opcion = Validador.leerEntero(sc, "Elija una opcion: ")
51     ;
52
53     // =====
54     // 3. CAPA DE SEGURIDAD Y MANEJO DE ERRORES (ROBUSTEZ)
55     // =====
56     // Todo el Switch-Expression se envuelve en un bloque
57     // try/catch global.
58     // Si cualquier operacion del menu o del servicio falla
59     // y lanza una excepcion,
60     // el flujo es capturado de inmediato. Esto evita que
61     // el programa "explote" (crash)
62     // y en su lugar muestra un mensaje elegante antes de
63     // reiniciar el bucle.
64     try
65     {
66         switch (opcion)
67         {
68             case 1 -> menu.agregarProducto();
69             case 2 -> menu.listarProductos();
70             case 3 -> menu.buscarProducto();
71             case 4 -> menu.actualizarProducto();
72             case 5 -> menu.eliminarProducto();
73             case 6 -> System.out.println("Hasta luego!");
74             default -> System.out.println("Opcion invalida.
75                 Elija un numero del 1 al 6.");
76         }
77     }
78     catch (ProductoNoEncontradoException |
79         StockInsuficienteException e)
80     {
81         // Captura errores especificos del dominio del
82         // negocio (ej. ID inexistente o falta de stock)
83         System.out.println("\n[ERROR DE NEGOCIO]: " + e.
84             getMessage());
85     }
86     catch (IllegalArgumentException e)
87     {
88         // Captura validaciones de datos genericos
89         // incorrectos (ej. campos vacios, valores
90         // negativos)
91         System.out.println("\n[ERROR DE VALIDACION]: " + e.
92             getMessage());
93     }
94 }

```

```

80
81 // =====
82 // 4. CONTROL VISUAL Y REFRESCO DE PANTALLA
83 // =====
84 // Detiene temporalmente la ejecucion para que los
85 // mensajes de exito o error
86 // no desaparezcan de inmediato. Se ejecuta siempre y
87 // cuando no se elija salir.
88 if (opcion != 6)
89 {
90     presionarEnterParaContinuar(sc);
91 }
92
93 while (opcion != 6); // Evalua la condicion de salida
94 // para ver si repite el ciclo
95
96 // Cierre preventivo del recurso Scanner al finalizar la
97 // ejecucion global del software
98 sc.close();
99
100 }
101
102 /**
103  * Metodo de Soporte: Alimenta la capa de servicios con
104  * productos quemados en codigo (mock data).
105  * @param service La instancia unica del servicio de productos
106  * donde se guardara la coleccion inicial.
107  */
108 private static void cargarDatosDePrueba(ProductoService service
109 )
110 {
111     service.guardar(new Producto("Cafe molido 500g", 4500, 30,
112     "Bebidas"));
113     service.guardar(new Producto("Yerba mate 1kg", 3200, 50, "
114     Bebidas"));
115     service.guardar(new Producto("Galletitas dulces", 1850,
116     100, "Almacen"));
117     service.guardar(new Producto("Aceite de oliva 500ml", 6700,
118     20, "Almacen"));
119     service.guardar(new Producto("Chocolate amargo 70%", 2900,
120     15, "Golosinas"));
121     System.out.println("Se cargaron 5 productos de prueba.\n");
122 }
123
124 /**
125  * Metodo de Soporte: Crea una pausa de consola obligando al
126  * usuario a presionar la tecla Enter.
127  * Esto evita que el menu secundario se encime velozmente sobre
128  * los resultados de las operaciones CRUD.
129  * @param sc La instancia compartida del Scanner de entrada.
130  */
131 private static void presionarEnterParaContinuar(Scanner sc)
132 {
133     System.out.println("\nPresione Enter para continuar...");

```

```

119         sc.nextLine(); // Bloquea la consola esperando unicamente
           el salto de linea del Enter
120     }
121 }

```

3.2. Validador.java

Listing 4: Clase Validador.java (Pre-entrega)

```

1  package util;
2
3  import java.util.InputMismatchException;
4  import java.util.Scanner;
5
6  import exception.StockInsuficienteException;
7
8  /**
9   * Clase Utilitaria (Utility Class): Provee metodos estaticos
        centralizados para validacion y lectura de datos.
10  *
11  * Sigue el patron Helper/Utility: Al declarar todos sus metodos
        como 'static', no requiere ser instanciada
12  * mediante el operador 'new'. Actua como una libreria interna que
        puede ser invocada directamente
13  * por cualquier capa de la aplicacion (UI o Service) bajo demanda.
14  */
15  public class Validador
16  {
17      // Nota de diseno: Idealmente, las clases utilitarias suelen
        llevar un constructor privado vacio
18      // para evitar explicitamente que alguien intente hacer:
        Validador v = new Validador();
19
20      // =====
21      // VALIDACIONES DE REGLAS DE NEGOCIO (Data Validation)
22      // =====
23      // Estos metodos aplican la tecnica de "Fail-Fast" (Fallar
        rapido). Si el parametro viola
24      // una restriccion, se interrumpe el hilo de inmediato lanzando
        una excepcion.
25      // No retornan nada (void): el silencio del metodo es sinonimo
        de que los datos son totalmente validos.
26
27      /**
28       * Valida que una cadena de texto contenga caracteres legibles
        y reales.
29       * @param nombre Cadena de texto a evaluar.
30       * @throws IllegalArgumentException Excepcion nativa si el
        texto es nulo o esta compuesto solo por espacios.
31       */
32      public static void validarNombre(String nombre)
33      {

```

```

34         // trim() remueve espacios fantasma en los extremos para
           evitar trampas como "   "
35     if (nombre == null || nombre.trim().isEmpty())
36     {
37         throw new IllegalArgumentException("El nombre no puede
           estar vacio.");
38     }
39 }
40
41 /**
42  * Valida que los valores monetarios sean financieramente
           logicos.
43  * @param precio Valor decimal a evaluar.
44  * @throws IllegalArgumentException Si el precio es menor a
           cero (los productos gratis con valor 0 son validos).
45  */
46 public static void validarPrecio(double precio)
47 {
48     if (precio < 0)
49     {
50         throw new IllegalArgumentException("El precio no puede
           ser negativo.");
51     }
52 }
53
54 /**
55  * Valida la existencia fisica del stock del inventario.
56  * @param stock Cantidad de unidades a evaluar.
57  * @throws StockInsuficienteException Excepcion personalizada
           utilizada para reflejar un error semantico del negocio.
58  */
59 public static void validarStock(int stock) {
60     if (stock < 0)
61     {
62         throw new StockInsuficienteException("El stock no puede
           ser negativo.");
63     }
64 }
65
66 /**
67  * Valida la categorizacion del catalogo.
68  * @param categoria Cadena de texto de la categoria.
69  * @throws IllegalArgumentException Si la categoria esta
           ausente o vacia.
70  */
71 public static void validarCategoria(String categoria)
72 {
73     // isBlank() evalua de forma optima si la cadena tiene
           longitud cero o puros espacios en blanco
74     if (categoria == null || categoria.isBlank())
75     {
76         throw new IllegalArgumentException("La categoria no
           puede estar vacia.");

```

```

77     }
78 }
79
80 // =====
81 // METODOS DE CAPTURA ROBUSTA POR CONSOLA (Console Input
82 // Processing)
83 // =====
84 // Resuelven el problema clasico del Scanner en Java: el
85 // desbordamiento o bloqueo del buffer
86 // cuando el usuario introduce caracteres no numericos o
87 // presiona saltos de linea imprevistos.
88
89 /**
90  * Captura de forma blindada un valor entero, atrapando errores
91  * tipograficos del usuario.
92  *
93  * @param sc Instancia unica y compartida del Scanner.
94  * @param mensaje Indicacion textual (Prompt) que guia al
95  * usuario sobre que ingresar.
96  * @return El numero entero ya validado sintacticamente.
97  */
98 public static int leerEntero(Scanner sc, String mensaje)
99 {
100     // Bucle infinito controlado: No se rompera hasta que el
101     // bloque 'try' alcance con exito el 'return'
102     while (true)
103     {
104         System.out.println(mensaje);
105         try
106         {
107             int valor = sc.nextInt();
108
109             // LIMPIEZA CLAVE DEL BUFFER: nextInt() consume el
110             // numero pero deja el caracter invisible
111             // del salto de linea ('\n') en la consola. sc.
112             // nextLine() consume ese residuo para evitar
113             // que el siguiente metodo de lectura se salte
114             // automaticamente.
115             sc.nextLine();
116
117             return valor; // Rompe el bucle e inmuniza la
118                 salida
119         }
120         catch (InputMismatchException e)
121         {
122             System.out.println("Debe ingresar un numero entero.
123                 Intente nuevamente.");
124
125             // RECOLECCION DE BASURA DEL BUFFER: Si el usuario
126             // escribe una letra (ej. "hola"),
127             // nextInt() falla y deja la palabra "hola"
128             // atrapada en el buffer. Si no llamamos a nextLine
129             // ()

```

```

116         // aqui, el bucle intentara leer "hola"
117         infinitamente provocando un ciclo sin fin.
118         sc.nextLine();
119     }
120 }
121
122 /**
123  * Captura de forma blindada un valor decimal, previniendo
124     fallas de localizacion por comas o puntos.
125  *
126  * @param sc Instancia compartida del Scanner.
127  * @param mensaje Indicacion en pantalla.
128  * @return El numero de punto flotante de doble precision (
129     double) validado.
130 */
131 public static double leerDouble(Scanner sc, String mensaje)
132 {
133     while (true)
134     {
135         System.out.println(mensaje);
136         try
137         {
138             double valor = sc.nextDouble();
139             sc.nextLine(); // Consume el residuo del salto de
140                 linea ('\n')
141             return valor;
142         }
143         catch (Exception e)
144         {
145             // Maneja genericamente cualquier anomalia
146                 tipografica (ej. ingresar un punto cuando el
147                 sistema espera coma)
148             System.out.println("Debe ingresar un numero decimal
149                 usando el separador correcto.");
150             sc.nextLine(); // Limpia la entrada corrupta del
151                 buffer antes de reiniciar el ciclo
152         }
153     }
154 }
155
156 /**
157  * Lee de forma directa lineas completas de caracteres sin
158     riesgo de saltos automaticos no deseados.
159  *
160  * @param sc Instancia compartida del Scanner.
161  * @param mensaje Indicacion en pantalla.
162  * @return La cadena de caracteres digitada por el usuario.
163  */
164 public static String leerTexto(Scanner sc, String mensaje)
165 {
166     System.out.println(mensaje);

```

```

159         // Al usar nextLine() directamente tras haber mantenido
        // limpios los buffers numericos anteriores,
160         // garantizamos una captura limpia sin efectos secundarios.
161         return sc.nextLine();
162     }
163 }

```

3.3. Producto.java

Listing 5: Clase Producto.java (Pre-entrega)

```

1 package model;
2
3 /**
4  * Modelo de Dominio o Plain Old Java Object (POJO): Representa la
5  * entidad basica de un Producto.
6  *
7  * Aplica el principio de Encapsulamiento: Los atributos son
8  * privados y el acceso se restringe
9  * a traves de metodos publicos (getters y setters). Esta clase
10  * sigue el Principio de Responsabilidad
11  * Unica (SRP), limitandose a modelar un objeto del mundo real; no
12  * gestiona bases de datos ni logica visual.
13  */
14 public class Producto
15 {
16     // =====
17     // ATRIBUTOS (Estado del Objeto)
18     // =====
19     // Al ser 'private', se protege la integridad de los datos
20     // impidiendo modificaciones
21     // externas maliciosas o directas que puedan romper el estado
22     // de la entidad.
23     private int id;
24     private String nombre;
25     private double precio;
26     private int stock;
27     private String categoria;
28
29     // =====
30     // CONSTRUCTORES (Instanciacion)
31     // =====
32
33     /**
34      * Constructor Parametrizado (Sin ID).
35      * Se utiliza para crear productos nuevos antes de ser
36      * persistidos. El ID no se incluye
37      * aqui porque su generacion es responsabilidad de la capa de
38      * persistencia (Base de datos / Servicio).
39      */
40     public Producto(String nombre, double precio, int stock, String
41         categoria)
42     {

```

```

34         this.nombre = nombre;
35         this.precio = precio;
36         this.stock = stock;
37         this.categoria = categoria;
38     }
39
40     /**
41     * Constructor Vacio (Por defecto).
42     * Requisito fundamental de la especificacion JavaBeans.
43     * Permite instanciar objetos "vacios"
44     * para completarlos progresivamente mediante Setters. Es
45     * obligatorio para que frameworks
46     * de mapeo como Hibernate/JPA, Jackson (JSON) o Spring puedan
47     * realizar la instanciacion por reflexion.
48     */
49     public Producto()
50     {
51         // Bloque intencionalmente vacio para cumplir con el
52         // estandar POJO/JavaBean
53     }
54
55     // =====
56     // METODOS DE ACCESO (Getters y Setters)
57     // =====
58     // Constituyen la interfaz publica segura para interactuar con
59     // los atributos privados.
60
61     public int getId()
62     {
63         return id;
64     }
65
66     public void setId(int id)
67     {
68         this.id = id;
69     }
70
71     public String getNombre()
72     {
73         return nombre;
74     }
75
76     public void setNombre(String nombre)
77     {
78         this.nombre = nombre;
79     }
80
81     public double getPrecio()
82     {
83         return precio;
84     }
85
86     public void setPrecio(double precio)

```

```

82     {
83         this.precio = precio;
84     }
85
86     public int getStock()
87     {
88         return stock;
89     }
90
91     public void setStock(int stock)
92     {
93         this.stock = stock;
94     }
95
96     public String getCategoria()
97     {
98         return categoria;
99     }
100
101     public void setCategoria(String categoria)
102     {
103         this.categoria = categoria;
104     }
105
106     // =====
107     // METODOS DE REPRESENTACION Y SALIDA
108     // =====
109
110     /**
111      * Sobreescritura del metodo estandar toString() heredado de la
112      * clase base java.lang.Object.
113      * Convierte la direccion de memoria nativa del objeto en una
114      * cadena de texto util y legible.
115      * Ideal para tareas de depuracion (debugging) o
116      * concatenaciones rapidas de texto.
117      * @return Una representacion formateada con el estado actual
118      * de los atributos del producto.
119      */
120     @Override
121     public String toString()
122     {
123         return "ID: " + id +
124             " | " + nombre +
125             " | $" + precio +
126             " | Stock: " + stock +
127             " | Categoria: " + categoria;
128     }
129
130     /**
131      * Envia directamente la informacion formateada del producto
132      * hacia el flujo de salida estandar (Consola).
133      * Diseñado específicamente para estandarizar la visualización
134      * de registros individuales dentro de la UI.

```

```

129     */
130     public void mostrar()
131     {
132         System.out.println("ID: " + id +
133             " | " + nombre +
134             " | $" + precio +
135             " | Stock: " + stock +
136             " | Categoria: " + categoria);
137     }
138 }

```

3.4. ProductoService.java

Listing 6: Clase ProductoService.java (Pre-entrega)

```

1  package service;
2
3  import java.util.ArrayList;
4  import java.util.List;
5
6  import exception.ProductoNoEncontradoException;
7  import model.Producto;
8  import util.Validador;
9
10 /**
11  * Capa de Servicio (Service Layer): Orquesta e implementa la
12   * Logica de Negocio (Business Logic) del sistema.
13  *
14  * Es la capa intermedia que actua como barrera de seguridad entre
15   * la Interfaz de Usuario (UI)
16   * y la persistencia de datos (en este caso, una coleccion en
17   * memoria).
18  *
19  * Responsabilidades:
20  * - Gestionar el ciclo de vida del almacen de datos (List).
21  * - Controlar la generacion e integridad de llaves primarias
22   * unicas (IDs).
23  * - Forzar el cumplimiento de las reglas o restricciones de
24   * negocio mediante validaciones antes de cualquier mutacion.
25  *
26  * Cumple con el principio de Desacoplamiento (Alta Cohesion / Bajo
27   * Acoplamiento): Al no contener codigo
28   * acoplado a la consola (sin Scanner ni System.out), esta misma
29   * clase podra ser reutilizada intacta en el futuro
30  * cuando el sistema migre a una arquitectura Web o API REST con
31   * Spring Boot.
32  */
33 public class ProductoService
34 {
35     // Almacenamiento volatil en memoria utilizando una lista
36     // dinamica (Estructura de Datos).
37     // Se encapsula como 'private' para que ninguna clase externa
38     // altere la coleccion sin pasar por las reglas del servicio.

```

```

29 private List<Producto> productos = new ArrayList<>();
30
31 // Generador secuencial de IDs unicos. Al ser un atributo
    estatico (static), su valor se conserva a nivel
32 // de Clase y no de Instancia, garantizando consistencia
    matematica y evitando la duplicidad de identificadores.
33 private static int contadorId = 1;
34
35 // =====
36 // OPERACIONES CRUD (Create, Read, Update, Delete)
37 // =====
38
39 /**
40  * CREATE: Procesa, valida y persiste una nueva entidad
    Producto en el sistema.
41  * Si alguna validacion de datos falla, se dispara una
    RuntimeException interrumpiendo el flujo,
42  * lo que garantiza que nunca ingresen datos corruptos a la
    lista.
43  *
44  * @param p Objeto producto transitorio (aun sin ID).
45  * @return El objeto Producto completamente persistido (con su
    ID oficial asignado).
46  */
47 public Producto guardar(Producto p)
48 {
49     // Fase de Validacion: Filtro estricto que evalua las
        restricciones del negocio antes de impactar los datos
50     Validador.validarNombre(p.getNombre());
51     Validador.validarPrecio(p.getPrecio());
52     Validador.validarStock(p.getStock());
53     Validador.validarCategoria(p.getCategoria());
54
55     // Asignacion de Identificador: Mecanismo de persistencia
        controlado internamente por el backend.
56     // Se aplica un post-incremento (contadorId++) para
        preparar automaticamente el valor del siguiente registro
        .
57     p.setId(contadorId);
58     contadorId++;
59
60     // Insercion en la estructura de datos en memoria
61     productos.add(p);
62
63     return p;
64 }
65
66 /**
67  * READ (Coleccion): Expone la lista completa de elementos
        registrados.
68  * @return List de objetos Producto actualmente almacenados en
        memoria.
69  */

```

```

70 public List<Producto> listarTodos()
71 {
72     return productos;
73 }
74
75 /**
76  * READ (Individual): Implementa un algoritmo de busqueda
77     lineal para localizar un registro por su ID unico.
78  *
79  * @param id Identificador numerico del producto deseado.
80  * @return El objeto Producto que coincide exactamente con el
81     parametro recibido.
82  * @throws ProductoNoEncontradoException Excepcion
83     personalizada si el bucle termina sin hallar coincidencias.
84  */
85 public Producto obtenerPorId(int id)
86 {
87     for (Producto p : productos)
88     {
89         if (p.getId() == id)
90         {
91             return p; // Retorno inmediato (Early Return) al
92                 encontrar el objetivo, optimizando el ciclo
93         }
94     }
95
96     // Control de errores de negocio: Si el flujo llega a este
97     punto, significa que el ID es invalido
98     throw new ProductoNoEncontradoException("No se encontro un
99     producto con el ID " + id);
100 }
101
102 /**
103  * UPDATE: Sobreescribe el estado de un registro existente
104     basandose en un ID de destino.
105  *
106  * Nota de Arquitectura: El parametro datos actua tecnicamente
107     como un DTO (Data Transfer Object),
108  * un contenedor temporal que transporta los nuevos datos desde
109     la UI hacia la logica de actualizacion.
110  *
111  * @param id Identificador del producto real que se desea
112     modificar.
113  * @param datos Objeto con la informacion actualizada que se va
114     a inyectar.
115  * @return La entidad Producto con su nuevo estado consolidado.
116  */
117 public Producto actualizar(int id, Producto datos)
118 {
119     // Reutilizacion de codigo: Se delega la busqueda a
120     obtenerPorId.
121     // Si el ID no existe, este metodo lanza su excepcion
122     nativa abortando el proceso de forma segura.

```

```

110         Producto p = obtenerPorId(id);
111
112         // Validamos exhaustivamente el paquete de cambios antes de
           mutar el objeto original
113         Validador.validarNombre(datos.getNombre());
114         Validador.validarPrecio(datos.getPrecio());
115         Validador.validarStock(datos.getStock());
116         Validador.validarCategoria(datos.getCategoria());
117
118         // Mutacion de estado por Referencia: Debido a que Java
           manipula las referencias a objetos en memoria,
119         // al alterar las propiedades de 'p', los cambios impactan
           directamente sobre el elemento dentro de la List.
120         // No es necesario remover e insertar el elemento de nuevo.
121         p.setNombre(datos.getNombre());
122         p.setPrecio(datos.getPrecio());
123         p.setStock(datos.getStock());
124         p.setCategoria(datos.getCategoria());
125
126         return p;
127     }
128
129     /**
130      * DELETE: Remueve fisicamente un registro del almacen de datos
           basandose en su ID.
131      * @param id Identificador numerico del objeto a dar de baja.
132      */
133     public void eliminar(int id)
134     {
135         // Localiza el objeto en memoria. Si no existe, detiene la
           operacion lanzando la excepcion correspondiente.
136         Producto p = obtenerPorId(id);
137
138         // Remocion fisica: Rompe el enlace del objeto dentro del
           ArrayList
139         productos.remove(p);
140     }
141 }

```

3.5. ProductoNoEncontradoException.java

Listing 7: Clase ProductoNoEncontradoException.java (Pre-entrega)

```

1 package exception;
2
3 /**
4  * Excepcion Personalizada de Negocio (Custom Domain Exception).
5  * Se dispara exclusivamente cuando un proceso del sistema intenta
           interactuar con un ID de producto inexistente.
6  *
7  * Arquitectura de Errores:
8  * Hereda de java.lang.RuntimeException, clasificandose como una
           Unchecked Exception (Excepcion No Verificada).

```

```

9  * A diferencia de las Checked Exceptions (que heredan directamente
    de java.lang.Exception), las excepciones no
10 * verificadas no obligan a propagar firmas con la palabra clave '
    throws' ni fuerzan el uso de bloques try/catch
11 * en todas las capas intermedias, manteniendo el código limpio y
    desacoplado.
12 *
13 * Ventajas del Diseño Expresivo (Domain-Driven Design):
14 * Provee una semántica clara y específica en la pila de llamadas (
    Stacktrace). Es mucho más descriptivo
15 * leer un error tipo 'ProductoNoEncontradoException' que uno
    genérico como 'IllegalArgumentException' o '
    NullPointerException'.
16 */
17 public class ProductoNoEncontradoException extends RuntimeException
18 {
19     /**
20     * Constructor de la excepción personalizada.
21     *
22     * @param mensaje Texto detallado que describe las
        circunstancias exactas de la anomalía (ej. el ID fallido).
23     */
24     public ProductoNoEncontradoException(String mensaje)
25     {
26         // La palabra clave 'super' invoca de forma explícita al
        constructor parametrizado de la clase
27         // base (RuntimeException). Esto inicializa los mecanismos
        nativos de Java para registrar el mensaje
28         // de error y congelar el estado actual del hilo,
        permitiendo luego recuperarlo mediante e.getMessage().
29         super(mensaje);
30     }
31 }

```

3.6. StockInsuficienteException.java

Listing 8: Clase StockInsuficienteException.java (Pre-entrega)

```

1  package exception;
2
3  /**
4   * Excepción Personalizada de Negocio (Custom Domain Exception).
5   * Se dispara cuando una operación viola las reglas de consistencia
        de inventario, ya sea al ingresar
6   * valores de stock físicamente imposibles (negativos) o al
        intentar egresar más unidades de las disponibles.
7   *
8   * Arquitectura de Errores:
9   * Al igual que 'ProductoNoEncontradoException', hereda de java.
        lang.RuntimeException, consolidándose
10 * como una Unchecked Exception (Excepción No Verificada). Esto
        permite un flujo libre de firmas en la capa

```

```

11  * de servicios ('ProductoService') y centraliza de forma elegante
    su captura en el controlador principal ('Main.java').
12  *
13  * Polimorfismo Semantico:
14  * Su diseno es lo suficientemente versatil para actuar en dos
    frentes del dominio del negocio:
15  * 1. Fase de Administracion (Backoffice): Previene errores de
    carga (ej. ingresar un stock inicial de -10).
16  * 2. Fase de Transaccion (E-Commerce): Servira como la base
    perfecta para el control del flujo cuando
17  * el sistema evolucione y requiera validar un Carrito de
    Compras (ej. un cliente pide 5 unidades pero quedan 2).
18  */
19  public class StockInsuficienteException extends RuntimeException
20  {
21      /**
22       * Constructor de la excepcion personalizada de stock.
23       *
24       * @param mensaje Texto descriptivo que detalla la
                inconsistencia de inventario detectada por el sistema.
25       */
26      public StockInsuficienteException(String mensaje)
27      {
28          // La palabra clave 'super' delega la inicializacion del
                mensaje al constructor de RuntimeException.
29          // Esto indexa la cadena de texto en las propiedades
                nativas del objeto arrojable (Throwable),
30          // quedando disponible para ser capturado en las capas
                superiores de la interfaz de usuario.
31          super(mensaje);
32      }
33  }

```

3.7. MenuProducto.java

Listing 9: Clase MenuProducto.java (Pre-entrega)

```

1  package ui;
2
3  import java.util.List;
4  import java.util.Scanner;
5
6  import model.Producto;
7  import service.ProductoService;
8  import util.Validador;
9
10 /*
11     Manejar la interaccion con el usuario: mostrar el menu, leer
        datos, mostrar mensajes de error o exito.
12
13     Responsable de:
14         - Mostrar el menu al usuario
15         - Pedirle los datos

```

```

16         - Mostrar los resultados
17
18     No contiene la logica del negocio.
19     No controla el flujo del programa.
20 */
21
22 public class MenuProducto
23 {
24     // Atributos de la clase: el Scanner y el Service se reciben
25     // por constructor (no se crean aqui dentro).
26     // Esto permite que quien instancie la clase tenga el control
27     // sobre que instancias usar.
28     private final Scanner sc;
29     private final ProductoService service;
30
31     /**
32      * Constructor de la clase.
33      * Implementa el patron de "Inyeccion de Dependencias por
34      * Constructor" (muy utilizado en Spring Boot).
35      * @param sc Objeto Scanner compartido para la lectura de datos
36      * por consola.
37      * @param service Capa de servicio encargada de la logica del
38      * negocio de los productos.
39      */
40     public MenuProducto(Scanner sc, ProductoService service)
41     {
42         this.sc = sc;
43         this.service = service;
44     }
45
46     /**
47      * Muestra visualmente las opciones disponibles en la consola.
48      * Incluye saltos de linea al inicio para limpiar y refrescar
49      * el aspecto de la pantalla.
50      */
51     public void mostrarMenu()
52     {
53         System.out.println("\n\n\n"); // Empuja el texto anterior
54         // hacia arriba para simular una pantalla limpia
55         System.out.println("===== TechLab - Gestion de Productos
56         =====");
57         System.out.println("1) Agregar producto");
58         System.out.println("2) Listar productos");
59         System.out.println("3) Buscar producto por ID");
60         System.out.println("4) Actualizar producto");
61         System.out.println("5) Eliminar producto");
62         System.out.println("6) Salir");
63         System.out.println("
64         =====");
65     }
66
67     // =====
68     // Operaciones del CRUD (Opciones del menu)

```

```

60 // =====
61
62 /**
63  * Opcion 1: Solicita los datos de un nuevo producto, lo
        construye y delega el guardado al servicio.
64  */
65 public void agregarProducto()
66 {
67     System.out.println("--- Nuevo producto ---");
68
69     // Se utiliza la clase utilitaria 'Validador' para
        garantizar que los tipos de datos ingresados sean
        correctos
70     String nombre = Validador.leerTexto(sc, "Nombre: ");
71     double precio = Validador.leerDouble(sc, "Precio: ");
72     int stock = Validador.leerEntero(sc, "Stock: ");
73     String categoria = Validador.leerTexto(sc, "Categoria: ");
74
75     // Construimos el objeto producto de forma temporal con los
        datos capturados
76     Producto p = new Producto(nombre, precio, stock, categoria)
        ;
77
78     // Enviamos el producto al servicio. El metodo 'guardar' se
        encarga internamente de validar las
79     // reglas de negocio (ej. precios no negativos) y de
        asignarle un ID unico de forma automatica.
80     Producto guardado = service.guardar(p);
81
82     // Confirmamos el exito de la operacion mostrando el ID
        generado por el sistema
83     System.out.println("Producto agregado con el id " +
        guardado.getId());
84 }
85
86 /**
87  * Opcion 2: Recupera la lista completa de productos desde el
        servicio y los muestra en pantalla.
88  */
89 public void listarProductos()
90 {
91     // Solicitamos la lista de productos almacenada en la capa
        de negocio
92     List<Producto> lista = service.listarTodos();
93
94     // Validacion local de interfaz: Si la lista viene vacia,
        informamos al usuario y cortamos la ejecucion del metodo
95     if (lista.isEmpty()){
96         System.out.println("No hay productos cargados.");
97         return;
98     }
99
100     System.out.println("--- Catalogo ---");

```

```

101         // Recorremos la lista usando un bucle for-each para
102         imprimir cada producto de manera estructurada
103         for(Producto p : lista){
104             p.mostrar(); // Llama al metodo interno del modelo
105             // disenado para imprimir con formato propio
106         }
107     }
108
109     /**
110     * Opcion 3: Solicita un ID numerico e intenta recuperar el
111     * producto correspondiente mediante el servicio.
112     */
113     public void buscarProducto()
114     {
115         // Solicitamos de forma segura un numero entero para el ID
116         int id = Validador.leerEntero(sc, "Ingrese el id del
117         producto: ");
118
119         // El servicio buscara el producto. Si no existe, lanzara
120         una excepcion (ProductoNoEncontradoException)
121         // que sera capturada y manejada por el try/catch ubicado
122         en la clase Main.java
123         Producto p = service.obtenerPorId(id);
124
125         // Si el producto fue encontrado con exito, se muestra su
126         representacion en texto
127         System.out.println("Encontrado: " + p);
128     }
129
130     /**
131     * Opcion 4: Busca un producto existente y actualiza todas sus
132     * propiedades con nuevos valores provistos.
133     */
134     public void actualizarProducto()
135     {
136         int id = Validador.leerEntero(sc, "Ingrese el id del
137         producto a actualizar: ");
138
139         // Recuperamos y mostramos primero los datos actuales para
140         que el usuario visualice que esta modificando.
141         // Al igual que en buscarProducto, si el ID no existe, el
142         flujo se interrumpe aqui por la excepcion del servicio.
143         Producto actual = service.obtenerPorId(id);
144         System.out.println("Datos actuales: " + actual);
145
146         System.out.println("--- Ingrese los nuevos datos ---");
147         String nombre = Validador.leerTexto(sc, "Nombre: ");
148         double precio = Validador.leerDouble(sc, "Precio: ");
149         int stock = Validador.leerEntero(sc, "Stock: ");
150         String categoria = Validador.leerTexto(sc, "Categoria: ");
151
152         // Creamos un nuevo objeto con los cambios requeridos

```

```

142     Producto datos = new Producto(nombre, precio, stock,
143                                   categoria);
144
145     // Enviamos el ID del destino y el objeto con las
146     // modificaciones al servicio para que procese el cambio
147     Producto actualizado = service.actualizar(id, datos);
148
149     // Confirmamos mostrando en pantalla el estado final del
150     // producto modificado
151     System.out.println("Producto actualizado: " + actualizado);
152 }
153
154 /**
155  * Opcion 5: Solicita un ID numerico y delega la remocion
156  * fisica o logica del producto al servicio.
157  */
158 public void eliminarProducto()
159 {
160     int id = Validador.leerEntero(sc, "Ingrese el id del
161     producto a eliminar: ");
162
163     // El servicio valida la existencia del ID y procede a
164     // borrarlo del almacen de datos
165     service.eliminar(id);
166
167     // Si el servicio no arrojo ninguna excepcion, confirmamos
168     // la eliminacion exitosa
169     System.out.println("Producto eliminado.");
170 }
171 }

```

4. Corridas

```

===== TechLab - Gestion de Productos =====
1) Agregar producto
2) Listar productos
3) Buscar producto por ID
4) Actualizar producto
5) Eliminar producto
6) Salir
=====
Elija una opcion:
1
--- Nuevo producto ---
Nombre:
Monster 500cc
Precio:
3500
Stock:
25
Categoria:
Bebidas
Producto agregado con el id 6
Presione Enter para continuar...

```

Figura 3: Elección de la opción 1

```

===== TechLab - Gestion de Productos =====
1) Agregar producto
2) Listar productos
3) Buscar producto por ID
4) Actualizar producto
5) Eliminar producto
6) Salir
=====
Elija una opcion:
2
--- Catalogo ---
ID: 1 | Cafe molido 500g | $4500.0 | Stock: 30 | Categoria: Bebidas
ID: 2 | Yerba mate 1kg | $3200.0 | Stock: 50 | Categoria: Bebidas
ID: 3 | Galletitas dulces | $1850.0 | Stock: 100 | Categoria: Almacen
ID: 4 | Aceite de oliva 500ml | $6700.0 | Stock: 20 | Categoria: Almacen
ID: 5 | Chocolate amargo 70% | $2900.0 | Stock: 15 | Categoria: Golosinas
ID: 6 | Monster 500cc | $3500.0 | Stock: 25 | Categoria: Bebidas
Presione Enter para continuar...

```

Figura 4: Elección de la opción 2

```

===== TechLab - Gestion de Productos =====
1) Agregar producto
2) Listar productos
3) Buscar producto por ID
4) Actualizar producto
5) Eliminar producto
6) Salir
=====
Elija una opcion:
3
Ingrese el id del producto:
3
Encontrado: ID: 3 | Galletitas dulces | $1850.0 | Stock: 100 | Categoria: Almacen
Presione Enter para continuar...

```

Figura 5: Elección de la opción 3

```

===== TechLab - Gestion de Productos =====
1) Agregar producto
2) Listar productos
3) Buscar producto por ID
4) Actualizar producto
5) Eliminar producto
6) Salir
=====
Elija una opcion:
4
Ingrese el id del producto a actualizar:
6
Datos actuales: ID: 6 | Monster 500cc | $3500.0 | Stock: 25 | Categoria: Bebidas
--- Ingrese los nuevos datos ---
Nombre:
Monster Mango Loco 500cc
Precio:
3000
Stock:
15
Categoria:
Bebida
Producto actualizado: ID: 6 | Monster Mango Loco 500cc | $3000.0 | Stock: 15 | Categoria: Bebida
Presione Enter para continuar...

```

Figura 6: Elección de la opción 4

```
===== TechLab - Gestion de Productos =====
1) Agregar producto
2) Listar productos
3) Buscar producto por ID
4) Actualizar producto
5) Eliminar producto
6) Salir
=====
Elija una opcion:
5
Ingrese el id del producto a eliminar:
1
Producto eliminado.
Presione Enter para continuar...

```

Figura 7: Elección de la opción 5

```
===== TechLab - Gestion de Productos =====
1) Agregar producto
2) Listar productos
3) Buscar producto por ID
4) Actualizar producto
5) Eliminar producto
6) Salir
=====
Elija una opcion:
6
Hasta luego!
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 03:24 min
[INFO] Finished at: 2026-10-06T21:24:47-03:00
[INFO] -----
josepaz@josepaz-Lenovo-IdeaPad-S145-15IGH:~/JAVA/java-preentrega$

```

Figura 8: Elección de la opción 6